# OpenClaw Primer: A Comprehensive, Podman-First Guide

28 May 2026

## Table of Contents

---

*Long-form edition · 28 May 2026*

---

# 1 Why This Primer Exists

When people first hear "OpenClaw," they can land on two completely different projects, and that ambiguity creates confusion before any technical work even starts. One historical usage points to an older game reimplementation, while the current, rapidly evolving project most practitioners mean is the OpenClaw AI assistant platform in the `openclaw/openclaw` repository. This document is explicitly about that modern assistant platform.

The second source of confusion is that the ecosystem around personal AI assistants has become noisy. Most guides either stay at marketing language or collapse into short install

checklists that do not prepare you for real operation. In practice, the first five minutes are not the hard part. The hard part begins when you need to choose model routing rules, define tool execution boundaries, safely expose channels, manage persistent state, and recover quickly when something fails.

This primer is written to bridge that gap. It is intentionally long-form and operationally grounded.

### 1.1 What This Covers

By the end, you should have three things: a clear mental model of what OpenClaw is, practical patterns for how people actually use it, and a container-first setup path that keeps the host surface area as small and explicit as possible.

## 2 What OpenClaw Is Used For

At its core, OpenClaw is a personal assistant control plane that sits between users, channels, models, and tools. That sounds abstract until you map it to daily use: it is the system that decides how your assistant receives a message, which model should handle it, what tools are allowed to run, and where the final response should be delivered.

This distinction matters because many early "assistant" systems are really single-route pipelines: one UI talking to one model endpoint with minimal policy. That is often fine until you need multiple channels, fallback behavior, tool governance, or long-lived assistant state. OpenClaw is used specifically when those requirements become real.

Another practical reason people choose OpenClaw is that it supports a local-first posture without forcing a local-only posture. You can run local providers as your primary path for cost and privacy while keeping hosted providers configured as fallback for resilience. In practice, this balance is often more useful than ideological purity in either direction.

### 2.1 What jobs OpenClaw performs in practice

In real deployments, OpenClaw handles channel ingress and egress, session routing, model selection, provider failover behavior, tool-call policy, and operational checks. It also carries lifecycle responsibilities that are easy to underestimate: configuration validation, diagnostics, health status, and continuity across restarts.

That is why it is better viewed as an operations layer for assistants, not as a simple chat surface.

### 2.2 What OpenClaw is not

OpenClaw is not a model server, and it does not replace model-serving systems such as Ollama, LM Studio, or vLLM. It is also not merely a themed chat UI. Its value comes from orchestration and control, not from owning the underlying inference engine.

## 3 How People Actually Use OpenClaw

Successful OpenClaw usage tends to follow a few repeatable deployment patterns. The common trait across these patterns is disciplined boundaries: clear model policy, clear channel policy, and clear tool policy.

### 3.1 Pattern A: Single-user daily assistant

This is the best starting point for most users. One gateway, one main assistant identity, one or two channels, and simple model fallback rules. The benefit is not merely simplicity; it is diagnosability. When behavior goes wrong, you can identify cause quickly because there are fewer moving parts.

In this mode, OpenClaw usually acts as a practical command center for drafting, summarization, lightweight automation, and recurring workflows. Teams that skip this phase often end up debugging avoidable complexity later.

### 3.2 Pattern B: Multi-channel command center

Once the single-user baseline is stable, many users extend to multiple surfaces: Control UI, mobile nodes, and one or more chat channels. This is where OpenClaw's channel and session model becomes powerful. The same assistant can remain coherent across different delivery paths while preserving context and policy.

The security posture must evolve with this transition. Pairing rules, allowlists, and non-main sandboxing become core controls rather than optional hardening.

### 3.3 Pattern C: Local-first with hosted safety net

This pattern is increasingly common because it aligns cost, privacy, and reliability in a practical way. Local providers handle primary traffic. Hosted providers remain available as fallback when local services are unavailable, slow, or unsuitable for the request.

The result is a system that is more private than hosted-only, more resilient than local-only, and usually cheaper than always using cloud models.

### 3.4 Pattern D: Containerized operations

Operators who care about reproducibility and controlled blast radius often run OpenClaw in containers, keep state on explicit mounts, and use host-side CLI as the management plane. This is the posture emphasized throughout this guide because it matches a self-contained operational objective.

## 4 Ideas for How You Could Use OpenClaw

The most useful ideas are concrete enough that you can implement a first version in days, not months.

For documentation-heavy work, OpenClaw can become a documentation operations assistant that summarizes long markdown, drafts release narratives, and enforces style conventions. The practical payoff is reduced documentation drift and better continuity across fast-moving engineering work.

For engineering triage, OpenClaw can classify issues, suggest duplicates, and route work by subsystem while remaining constrained by explicit tool policy. Used carefully, it can reduce intake chaos without granting broad automation authority.

For personal research, OpenClaw can act as a persistent synthesis layer. The value is less about one perfect answer and more about retained context under storage boundaries you control.

For homelab operations, it can aggregate health checks and logs into digestible operational summaries. Even modest setups benefit when low-level telemetry becomes readable status rather than raw noise.

For role-separated workflows, OpenClaw can host multiple assistant identities with distinct workspaces and policy. That separation can drastically reduce accidental cross-context behavior.

## 5 OpenClaw Architecture in One Mental Model

A practical debugging model is to think in layers: gateway, agent, provider, execution, state. Most troubleshooting becomes easier when you identify the failing layer before changing configuration.

The gateway layer handles ingress, routing, APIs, and session plumbing. The agent layer carries prompt context, model selection logic, and tool-call behavior. The provider layer

maps to model-serving endpoints and auth behavior. The execution layer is where tools run, either on host or sandbox. The state layer holds long-lived truth: config, auth profiles, session data, and workspace.

This layered view prevents category errors. A provider timeout is not a channel policy problem. A risky tool action is usually an execution-policy issue, not a model quality issue. A restart regression is often state drift, not immediate runtime logic.

### 5.1 State locations that matter

In container-first setups, state discipline is non-negotiable. Configuration, auth profile material, workspace data, and session artifacts should all persist outside ephemeral container layers. If this boundary is unclear, upgrades and restores become fragile.

## 6 Comprehensive Local Setup (Podman-First, Self-Contained)

This section is intentionally operational and assumes your goal is repeatable operation, not one-time demonstration.

### 6.1 Deployment goals

A strong target posture is rootless Podman runtime, explicit state persistence mounts, minimal host dependencies, and optional user-level service management for restart behavior. This keeps host contracts narrow while preserving operational control.

### 6.2 Prerequisites

You need Linux, rootless Podman, OpenClaw CLI on host, and optionally `systemd --user` for service management. On headless systems, lingering can be used for boot-time continuity.

### 6.3 Bootstrapping flow

Use source checkout to align with official helper scripts.

```
git clone https://github.com/openclaw/openclaw.git
cd openclaw
```

Initialize Podman path:

```
./scripts/podman/setup.sh
```

Launch runtime:

```
./scripts/run-openclaw-podman.sh launch
```

Run onboarding in container context:

```
./scripts/run-openclaw-podman.sh launch setup
```

Access dashboard:

- `http://127.0.0.1:18789/`

Operate via host CLI targeting the container:

```
export OPENCLAW_CONTAINER=openclaw
openclaw gateway status --deep
openclaw dashboard --no-open
```

### 6.4 Persistence model

Treat persistence as architecture, not convenience. Config, workspace, auth, and session artifacts should all map to known durable paths. Avoid anonymous state where possible.

### 6.5 Optional Quadlet mode

If you need service semantics and restart behavior, user-level Quadlet can provide cleaner day-2 operations than manual relaunch loops.

### 6.6 Day-2 operations

```
podman logs -f openclaw
podman stop openclaw
./scripts/run-openclaw-podman.sh launch
openclaw gateway status --deep
openclaw doctor
```

### 6.7 Ollama-native quick setup

The Podman bootstrapping flow above is the self-contained posture this guide emphasizes, but there is a faster path for users who already run Ollama and want a single-command launch. Ollama can drive OpenClaw directly, handling installation, model selection, and daemon startup in one step.

```
ollama launch openclaw
```

On first run this walks you through the full interactive setup:

1. Installing OpenClaw via npm if it is not already present.
2. A security notice explaining the tool-level access the agent will be granted.
3. Model selection — local or cloud.
4. Configuring your messaging provider(s) and starting the gateway daemon.

For unattended starts — boot-time services or container launch — use the headless variant. `--yes` skips the interactive prompts and `--model` is required:

```
ollama launch openclaw --model qwen3.5 --yes
```

To stop the gateway:

```
openclaw gateway stop
```

This path trades some of the explicit container boundaries described above for convenience. It is a good fit for single-user local setups where you control the host directly.

### 6.8 Recommended Adoption Sequence

1. Bring up Podman runtime and verify health.
2. Configure one local provider first.
3. Add one hosted fallback.
4. Enable non-main sandboxing before opening external channels.
5. Containerize model services for stronger containment if needed.
6. Establish backup cadence.

## 7 Running OpenClaw with Local LLMs

OpenClaw integrates with both native local providers and OpenAI-compatible proxy-style providers. Choosing between them is primarily about behavior guarantees and operational preference.

### 7.1 Model selection and fallback semantics

OpenClaw distinguishes configured defaults, auto-selected fallback state, and explicit user overrides. This is operationally important. Configured defaults can walk fallback chains. Explicit user selections are strict by design and fail visibly when unavailable.

Model choice is not just a quality decision; it is a context-budget decision. OpenClaw is an agentic assistant that does multi-turn reasoning, calls tools, and processes long context, so the local model you pick needs room to work. Plan on at least a 64K token context window for local models running agentic workloads. Agent loops accumulate tool call results, conversation history, and web search output into context very quickly, and a model that cannot hold that working set will start truncating or failing mid-task.

The following models are practical defaults for local-first operation, with two cloud entries kept as fallback:

| Model | VRAM needed | Notes |
|---|---|---|
| `qwen3.5` (local) | ~11 GB | Reasoning, coding, vision — the local sweet spot |
| `gemma4` (local) | ~16 GB | Strong reasoning and code |

| Model | VRAM needed | Notes |
|---|---|---|
| `qwen3.5:cloud` | None local | Falls back to Ollama cloud; good for testing |
| `kimi-k2.5:cloud` | None local | Multimodal reasoning with sub-agents |

### 7.2 Ollama

Ollama is a strong local-first path, but the key setup detail is API mode. For OpenClaw's Ollama provider, native API endpoint behavior is preferred over `/v1` compatibility mode when reliable tool behavior matters.

```
ollama pull gemma4
export OLLAMA_API_KEY="ollama-local"
openclaw onboard
openclaw models list --provider ollama
openclaw models set ollama/gemma4
```

### 7.3 LM Studio

LM Studio is useful when you want local model serving with easier lifecycle controls. OpenClaw can target LM Studio with OpenAI-compatible request modes depending on capability.

### 7.4 vLLM

vLLM is commonly used for higher-throughput serving scenarios. In OpenClaw, it is treated as an OpenAI-compatible provider and should be configured with explicit timeout and model metadata assumptions.

### 7.5 LiteLLM

LiteLLM is valuable as an abstraction and routing layer over multiple model backends. It is often used where centralized policy and provider switching are required.

### 7.6 On-demand local services

OpenClaw can also manage provider-local service startup via `localService` config, allowing heavyweight model services to spin up on demand instead of running continuously.

### 7.7 Constrained hardware: partial GPU offloading

The reason to run this on constrained hardware at all is not raw speed — you will not beat a hosted model on tokens per second. The benefit is privacy and persistence: local files, local

databases, and MCP-connected tools, all under boundaries you own. On a 64 GB RAM / 6 GB VRAM laptop this is enough to run a practical roaming assistant with large retained context, which is often more valuable day-to-day than a faster model that forgets everything between sessions.

That class of laptop cannot fully host the recommended OpenClaw models in 6 GB of VRAM, but 64 GB of RAM is plenty for a strong partial-offload setup: put as many layers as possible on the GPU and keep the rest on CPU/RAM.

With llama.cpp, the `--n-gpu-layers` flag controls how many transformer layers go to the GPU. A 7B model has 32 layers; a 13B has 40. Loading 28 of 32 layers of a 7B Q4 model typically uses ~3.5–4 GB of VRAM, leaving KV-cache headroom, with the remaining layers on CPU. Your 64 GB of RAM is what absorbs large KV-cache growth during long agent loops — that headroom is the whole point.

With Ollama, set the equivalent through a Modelfile:

```
cat > ~/qwen-laptop.Modelfile << 'EOF'
FROM qwen2.5:7b-instruct-q4_K_M
PARAMETER num_gpu 28
PARAMETER num_ctx 65536
PARAMETER num_thread 8
EOF

ollama create qwen-laptop -f ~/qwen-laptop.Modelfile
ollama launch openclaw --model qwen-laptop
```

Expect roughly 8–15 tok/s for 7B Q4 with partial offload on a modern Intel/AMD laptop. Long-context prefill is slower, but interactive chat stays usable. Treat `num_thread 8` as a starting point and tune toward your physical core count — too many threads adds overhead rather than throughput.

## 8 Podman + Local LLMs: Containment Patterns

There are three practical containment patterns.

Pattern one runs OpenClaw in containers but leaves model services on host. It is easy to adopt but less self-contained. Pattern two containerizes both gateway and model services with explicit persistence paths, which is often the best balance of containment and operability. Pattern three adds stricter sandboxing and narrow tool policies for higher-risk surfaces.

Most mature setups converge toward pattern two after proving behavior in pattern one.

# 9 Example Configuration Snippets

## 9.1 Local-first with hosted fallback

```
{
  agents: {
    defaults: {
      model: {
        primary: "ollama/gemma4",
        fallbacks: ["anthropic/claude-sonnet-4-6"]
      }
    }
  },
  models: {
    mode: "merge",
    providers: {
      ollama: {
        baseUrl: "http://ollama:11434",
        api: "ollama",
        apiKey: "ollama-local",
        timeoutSeconds: 300,
        models: [
          {
            id: "gemma4",
            name: "gemma4",
            reasoning: false,
            input: ["text"],
            cost: { input: 0, output: 0, cacheRead: 0, cacheWrite: 0 },
            contextWindow: 32768,
            maxTokens: 4096
          }
        ]
      }
    }
  }
}
```

## 9.2 Non-main sandbox baseline

```
{
  agents: {
    defaults: {
      sandbox: {
        mode: "non-main",
        scope: "agent",
        workspaceAccess: "none"
      }
    }
  }
}
```

## 9.3 Generic OpenAI-compatible local provider

```
{
  agents: {
    defaults: {
      model: { primary: "local/my-model" }
    }
  },
  models: {
    mode: "merge",
```

```
    providers: {
      local: {
        baseUrl: "http://127.0.0.1:8000/v1",
        apiKey: "sk-local",
        api: "openai-completions",
        timeoutSeconds: 300,
        models: [
          {
            id: "my-model",
            name: "my-model",
            reasoning: false,
            input: ["text"],
            cost: { input: 0, output: 0, cacheRead: 0, cacheWrite: 0 },
            contextWindow: 120000,
            maxTokens: 8192
          }
        ]
      }
    }
  }
}
```

# 10 Operational Reference

## 10.1 Security and Hardening Checklist

Hardening should scale with exposure. Loopback-only personal setups can prioritize convenience. Any remotely reachable surface should prioritize strict channel policy, controlled tool access, and sandbox boundaries.

At minimum, keep publish scope narrow, enforce pairing and allowlists, avoid broad host binds, and run diagnostics after significant config changes.

## 10.2 Troubleshooting Guide

Start with transport and state truth before tuning behavior. Reachability failures usually come from runtime/port/publish issues. Auth failures usually come from token mismatch or target confusion. Provider mismatches often come from namespace assumptions in containerized environments.

When tool calls appear as plain text, treat backend compatibility as a likely cause before rewriting assistant logic.

## 10.3 Hardening Profile Matrix

| Control Area | Dev | Trusted-Home | Internet-Exposed |
|---|---|---|---|
| Publish scope | loopback | loopback + controlled remote access | loopback + authenticated proxy/tailnet |
| Channel policy | minimal | pairing + allowlists | strict pairing + strict allowlists |
| Sandbox mode | off/non-main | non-main | all or tightly scoped non-main |
| Workspace access | rw acceptable | prefer none/ro | none by default |
| Tool policy | broad for testing | constrained | deny-by-default for risky tools |
| Fallback strategy | simple | explicit chain | explicit chain + active monitoring |
| Backup policy | ad hoc | scheduled | scheduled + off-host encrypted retention |

Do not advance to a higher exposure profile until the current profile is stable and validated.

## 11 Reference Links

- OpenClaw repository: https://github.com/openclaw/openclaw
- OpenClaw docs: https://docs.openclaw.ai
- Podman install guide: https://docs.openclaw.ai/install/podman
- Docker guide: https://docs.openclaw.ai/install/docker
- Models: https://docs.openclaw.ai/concepts/models
- Model failover: https://docs.openclaw.ai/concepts/model-failover
- Local models: https://docs.openclaw.ai/gateway/local-models
- Local model services: https://docs.openclaw.ai/gateway/local-model-services
- Sandboxing: https://docs.openclaw.ai/gateway/sandboxing
- Ollama provider: https://docs.openclaw.ai/providers/ollama
- LM Studio provider: https://docs.openclaw.ai/providers/lmstudio
- vLLM provider: https://docs.openclaw.ai/providers/vllm
- LiteLLM provider: https://docs.openclaw.ai/providers/litellm

# 12 Runbooks

## 12.1 Full Podman Compose Stack (OpenClaw + Ollama + Optional vLLM)

This project includes a concrete compose baseline so the primer is directly actionable. The stack is designed for local-only exposure, explicit persistence, and optional model-serving expansion.

### 12.1.1 Included operational files

- `podman-compose.yml`
- `scripts/backup_state.sh`
- `scripts/restore_state.sh`

### 12.1.2 Launch flow

```
podman compose -f podman-compose.yml up -d
podman compose -f podman-compose.yml ps
podman compose -f podman-compose.yml logs -f openclaw
```

### 12.1.3 Optional vLLM profile

```
podman compose -f podman-compose.yml --profile vllm up -d
```

## 12.2 Backup and Restore

State integrity is central to reliable assistant operation. The included scripts provide a baseline snapshot and restore workflow.

### 12.2.1 Backup

```
./scripts/backup_state.sh
```

### 12.2.2 Restore

```
./scripts/restore_state.sh ./backups/openclaw_state_YYYYMMDD_HHMMSS.tar.gz
```

### 12.2.3 Post-restore validation

```
podman compose -f podman-compose.yml up -d
openclaw gateway status --deep
openclaw models status
openclaw models list --provider ollama
```

## 12.3 Deterministic Bring-Up Sequence

```
podman compose -f podman-compose.yml up -d
export OPENCLAW_CONTAINER=openclaw
openclaw onboard
openclaw models status
openclaw config set agents.defaults.sandbox.mode '"non-main"'
openclaw gateway status --deep
./scripts/backup_state.sh
```

## 13 Closing Perspective

The real value of this stack is not simply running local models. It is controlling assistant behavior under explicit operational rules you own. If you maintain clear boundaries for runtime, state, policy, and recovery, OpenClaw can move from "interesting tool" to dependable daily system.

No deployment is literally zero-touch. The real goal is explicit host contracts, explicit persistence, explicit secret handling, and explicit recovery steps. OpenClaw plus rootless Podman fits this model well when boundary discipline is maintained.